MAD 1 WEEK 3

L3.1: Overview of MVC

Model-View-Controller (MVC):

- Originates from Smalltalk-80, an object-oriented programming language.
- Developed by Xerox PARC, a research and development company.
- A design pattern commonly used in software development.
- Components:
    - **Model:** Represents the application object and data logic.
    - **View:** Handles the presentation and UI representation.
    - **Controller:** Manages user input and updates the model and view accordingly.

L3.2: Views

User Interface (UI):

- UI can be represented as:
    - Screen (Graphical UI)
    - Audio (Voice-based UI)
    - Vibration (Haptic feedback)
- User interaction is determined by hardware constraints and target devices.
- User-Agent information helps identify the user's context.
- Input methods can include:
    - Keyboard/Mouse
    - Touchscreen
    - Spoken voice
    - Custom buttons

Types of Views:

1. **Fully Static:** No user interaction (e.g., Brochure website, Landing page).
2. **Partly Dynamic:** Some interaction (e.g., Google search, News websites).
3. **Mostly Dynamic:** High interaction (e.g., Social media, E-commerce websites).

Output Formats:

- Direct rendering (HTML, dynamic images)
- Machine-readable formats (JSON/XML)
- Views serve as representations useful to other entities.

L3.3: User Interface Design

Design Goals:

- **Simple:** Easy to understand and use.
- **Efficient:** Users achieve their goals with minimal effort.
- **Aesthetic Appeal:** Enhances user experience.

Key Principles of UI Aesthetics:

- Simplicity
- Consistency
- Visual Hierarchy
- Use of Color
- Use of Typography
- Use of Graphics

Accessibility Considerations:

- Text alternatives for non-text content.
- Screen reader compatibility.
- Proper color contrast for readability.

Systematic UI Design Process:

1. Functionality requirements gathering.
2. User and task analysis.
3. Prototyping (Wireframes & Mockups).
4. Testing (Usability & Accessibility).

L3.4: Usability Heuristics (Jakob Nielsen's 10 Principles):

1. **Visibility of System Status:** Inform users about ongoing processes.
2. **Match Between System and Real World:** Use familiar language and concepts.
3. **User Control and Freedom:** Provide undo and redo options.
4. **Consistency and Standards:** Follow platform conventions.
5. **Error Prevention:** Reduce potential user errors.
6. **Recognition Rather Than Recall:** Make options visible.
7. **Flexibility and Efficiency of Use:** Provide shortcuts and customization.
8. **Aesthetic and Minimalist Design:** Avoid unnecessary information.
9. **Help Users Recognize, Diagnose, and Recover from Errors:** Use clear error messages.

10. **Help and Documentation:** Provide easily accessible and searchable documentation.

L3.5: Tools - Part 1 (Wireframes)

- A wireframe is a skeletal representation of a user interface.
- It includes:
  - Information Design
  - Navigation Design
  - User Interface Layout

L3.6: Tools - Part 2 (Programmatic HTML Generation)

PyHTML: A Python library for generating HTML code.

- Installation: `$ pip install pyhtml`
- Example Usage:

```
import pyhtml as h
template = h.html(
    h.body(
        h.h1('Hello, World!'),
        h.p('This is a paragraph'),
        h.div(
            h.img(src='https://picsum.photos/200/300'),
            h.a('Click here', href='https://www.google.com')
        )
    )
)
print(template.render())
```

Jinja2: A Python templating engine used in Flask and Django.

- Installation: `$ pip install Jinja2`
- Example Usage:

```
from jinja2 import Template
template = Template('Hello {{ name }}!')
print(template.render(name='User'))
```

L3.7: Accessibility

Forms of Disabilities:

- Visual
- Speech
- Touch
- Sensor-Motor

W3 Web Content Accessibility Guidelines (WCAG):

- **Perceivable:** Provide alternatives for non-text content.
- **Operable:** Ensure keyboard accessibility.
- **Understandable:** Maintain clarity and predictability.
- **Robust:** Support assistive technologies.

Principles for Accessible UI Design:

- Provide captions for multimedia.
- Avoid content that causes seizures.
- Enable easy navigation.
- Ensure compatibility with future technologies.

Evolution of Google Icons:

- Google's icons have evolved to reflect simplicity and modern aesthetics.
- Early icons were detailed and skeuomorphic (real-world resemblance).
- Transitioned to flat design for clarity and efficiency.
- Material Design introduced consistency and adaptability.
- Current icons prioritize minimalism and accessibility.

This document provides a structured reference for understanding MVC, UI design, usability heuristics, tools, and accessibility principles, along with a brief overview of the evolution of Google's icons.

L3.8: Style Guides

Definition:

- A style guide is a collection of design standards and best practices used to create a cohesive UI/UX across an application or platform.

Common UI Style Guide Components:

1. **Typography:** Font families, sizes, and spacing.
2. **Color Palette:** Primary, secondary, and neutral colors.
3. **Buttons & Forms:** Consistency in interaction elements.
4. **Icons & Imagery:** Standard iconography and visual elements.
5. **Grid & Layout System:** Defines structure and spacing.

Examples of Popular Style Guides:

- Google Material Design
- Apple Human Interface Guidelines
- Microsoft Fluent Design

Benefits of Using Style Guides:

- Ensures consistency across the UI.
- Speeds up the development process.
- Improves user experience and accessibility.
- Makes collaboration easier between designers and developers.

Screencast:

How to Import `sys`?

To use the `sys` module, first import it:

```
import sys
```

Key Features of `sys` Module

1. Command-Line Arguments (`sys.argv`)

- `sys.argv` stores command-line arguments as a list.
- `sys.argv[0]` is the script name, and subsequent elements are arguments.

Example:

```
import sys
print("Script Name:", sys.argv[0])
```

```
if len(sys.argv) > 1:
    print("Arguments:", sys.argv[1:])
```

Run this script in the terminal:

```
python script.py arg1 arg2
```

Exiting the Program (`sys.exit()`)

Use `sys.exit(code)` to stop execution. The `code` (default `0`) indicates success (`0`) or failure (`1` or other numbers).

Example:

```
import sys
print("Exiting the program...")
sys.exit(0)  # Exits the script
print("This line will not execute.")
```

Basic Flask app:

```
from flask import Flask, render_template

app = Flask(__name__)

@app.route("/")
def home():
    return render_template("home.html")

if __name__ == "__main__":
    app.run(debug=True)
```

Basic jinja:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Home</title>
</head>
<body>
    <h1>Welcome to Flask</h1>
    <p>Hello, {{ name }}! Welcome to our first Flask app with Jinja.</p>
</body>
</html>
```

How to pass names?

```
@app.route("/")
def home():
    return render_template("home.html", name="Student")
```

Loops in jinja:

```
<h2>Your Favorite Fruits</h2>
<ul>
    {% for fruit in fruits %}
        <li>{{ fruit }}</li>
    {% endfor %}
</ul>
```

flask:

```
@app.route("/")
```

```python
def home():
    fruits = ["Apple", "Banana", "Cherry"]
    return render_template("home.html", name="Student", fruits=fruits)
```

Conditional in jinja:

```
{% if name == "Student" %}
    <p>Welcome back, {{ name }}!</p>
{% else %}
    <p>Who are you?</p>
{% endif %}
```

Template inheritance:

Create base.html:

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>{% block title %}My Flask App{% endblock %}</title>
</head>
<body>
    <header>
        <h1>My Flask App</h1>
    </header>

    <main>
        {% block content %}{% endblock %}
    </main>
</body>
</html>
```

Home.html:

```
{% extends "base.html" %}

{% block title %}Home{% endblock %}

{% block content %}
    <h2>Welcome to the Home Page</h2>
    <p>Hello, {{ name }}!</p>
{% endblock %}
```